

Dynamic Bond Index Replication using XGBoost and Kalman Filters

Author: Abhinandan Shrivastava

Imperial Business School

MSc Risk Management and Financial Engineering

[LinkedIn Profile](#)

[GitHub Profile](#)

Contents

1	Client Specification	1
2	Introduction	2
2.1	Data Exploration	2
2.1.1	TRACE Dataset	2
2.1.2	Bloomberg US Corporate Bonds Index	4
3	Literature Review	4
4	Quantitative Methodology	5
4.1	Rolling Window Evaluation	6
4.2	XGBoost	7
4.3	Kalman Filter	8
5	Model Evaluation	9
5.1	Tracking Errors	10
5.2	Correlation Analysis	11
6	Conclusions	12
6.1	Challenges	13
	Appendix A1: Plots	14
	Appendix A2: Code	15

List of Tables

1	Rolling Window Logic	6
2	Tracking Error Summary Statistics (%)	10
3	Portfolio Returns Summary (% , annualized)	10

List of Figures

1	Distribution of Bond CUSIPs	3
2	Corporate Bonds Outstanding	3
3	Functional Flow	5
4	Bonds Held Overtime: XGB	9
5	Bonds Held Overtime: Kalman	9
6	In-Sample Tracking Errors: XGB vs KLM	10
7	OOS Tracking Errors: XGB vs KLM	11
8	Correlation Analysis: XGB	11
9	Correlation Analysis: Kalman	12
10	Corporate Bonds Outstanding (Sliced)	14
11	Bloomberg Index Plots	14
12	Tracking Errors Overtime	14
13	Tracking Errors Distribution	15

1 Client Specification

Jappsar Asset Management Ltd. is a London-based firm which uses cutting-edge quantitative and AI-driven methodologies to deliver a range of products to its High Net Worth (HNI) clients. The products include a variety of actively managed mutual funds and passive ETFs, all of which are spread across various asset classes like equity, fixed income, currencies, and commodities.

Amongst its multiple products, the fixed income vertical at Jappsar offers the firm's flagship product; the US Corporate Credit ETF, which closely tracks Bloomberg's Corporate Bond Index (*ticker: LUACTRUU*), and is an instrument used by many investors to gain passive exposure to the fixed income markets.

The Bloomberg index comprises of 4500 bonds on average, and replicating the index by buying all the bonds is impractical due to liquidity constraints, transaction costs, and operational infeasibility. Sarah Green, Head of Fixed Income Products at Jappsar has tasked us with tackling this problem. This project will investigate quantitative strategies that handles this task by creating a portfolio with a subset of bonds, which will track the index with minimal tracking error.

The tasks of this project are:

1. Propose and implement portfolio construction methodologies which deem fit for the task at hand.
2. Evaluate model performance in terms of tracking error, and correlation with the index .
3. Recommend Sarah robust quantitative strategies, while covering the pros and cons of each comprehensively.

The following project hence aims to replicate the Bloomberg index as efficiently as possible, with practical considerations in mind.

2 Introduction

Passive investing has become really popular, especially since the 2000s. By the end of 2023, the Assets Under Management (AUM) for passive funds hit \$13.29 trillion, officially surpassing the AUM for active funds, as reported in In the same report, we can observe a strong trend of passive equity products dominating their fixed income counterparts. Although this trend is still prevalent, a growing number of investors have started using fixed income ETFs to gain passive exposure to the bond market.

This project is intended to be helpful for managers who create and maintain such ETF products. Unlike their equity counterparts, the bond market (and subsequently, bond indices) faces a few unique challenges, such as:

1. High Turnover

Bonds always have a maturity, so unlike stocks, bonds cease to exist after some time. The ETF needs to constantly replace expired bonds with new issues in order to successfully maintain itself.

2. Investable Universe

A company typically has only one stock offering (at most two or three, if we include non-voting and preference shares). However, a company may have 5–8 different types of bond offerings active at any given time. This creates a challenge for any ETF manager, as too many bonds contribute towards the index.

3. Liquidity Constraints

Since bonds are usually traded Over-The-Counter (OTC), many are only liquid when newly issued. Over their lifetime, a lot of bonds are not traded frequently, raising liquidity concerns.

Due to these issues, managers maintaining fixed income ETFs face a great challenge, and this project aims to address a few of them. In a crux, this project will try to replicate Bloomberg’s US Corporate Credit Index while buying as few bonds as possible. A strong replication approach will be followed, where the aim is to track the returns of the index. [Section 2](#) continues by describing the datasets used for this exercise. [Section 3](#) covers leading academic work that has been done in this field. [Section 4](#) explains the methodology followed. [Section 5](#) outlines the results obtained by the project. Lastly, [Section 6](#) summarizes the conclusions and challenges of this project. [Appendix A1: Plots](#) contains additional plots, and [Appendix A2: Code](#) appends the code used to implement this project.

2.1 Data Exploration

This project utilizes two main datasets: Bloomberg’s US Corporate Bond Index, and the TRACE dataset for constituent corporate bonds. TRACE stands for Trade Reporting and Compliance Engine, and contains data on bonds which are transacted OTC.

2.1.1 TRACE Dataset

As TRACE is transaction data, it is usually quite messy and tough to handle. There are many missing values, bogus returns due to data entry errors, and other erroneous values. Apart from this, the scale of the dataset is massive: the raw dataset had about 1.3 million rows, which created challenges in terms of computation.

A cleaned version of the TRACE dataset was uploaded to [Stolborg \(2023\)](#), which is used in

this project. The dataset spans July 2002 - November 2021, and is of monthly frequency. Below are a few characteristics of this unique dataset.

The entire dataset has 38,877 unique bonds, all of which are tracked by their respective CUSIPs, which is a 9-digit alphanumeric code uniquely identifying all these bonds. Studying these CUSIPs gives insights on how many bonds were available overtime, and for how long.

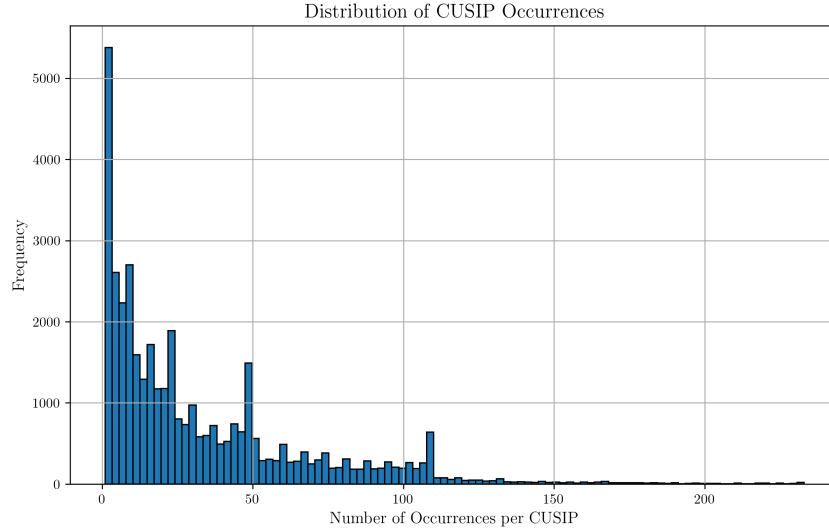


Figure 1: Distribution of Bond CUSIPs

Figure 1 plots the number of occurrences for each bond, displaying average maturity in months for the bonds. The bonds had an average occurrence of 36, meaning a maturity of 3 years on average. A lot of bonds have single-digit occurrences, which will need proper processing.

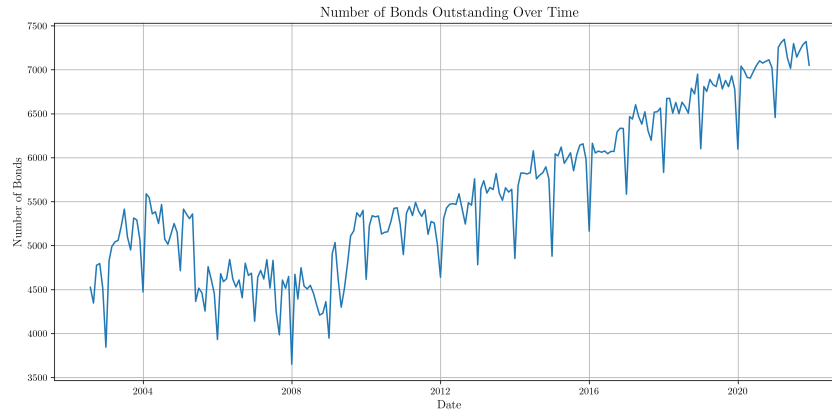


Figure 2: Corporate Bonds Outstanding

Figure 2 cross-sectionally plots unique bonds available each month, translating into the investment universe for the portfolio. An obvious pattern is visible; the number of outstanding bonds always drops at the end of the calendar year. **Figure 10** in **Appendix A1: Plots** slices this plot to study this effect closely. This feature might be unique to this dataset, as a slowdown in trading activity is expected as the year closes. Due to this, many bonds will not record a transaction, leading to the observable drops.

2.1.2 Bloomberg US Corporate Bonds Index

Bloomberg's US Corporate Bond Index (*ticker: LUACTRUU*) is the index the project will try to replicate. The index data was fetched according to the dates of the TRACE dataset. The Excel Add-In on Bloomberg's Terminal [Bloomberg L.P. \(n.d.\)](#) was used to query the data.

[Figure 11](#) in [Appendix A1: Plots](#) are the monthly price and return plots respectively. The index has grown modestly overtime, but has not been extraordinary. The monthly returns series looks stationary, with outliers being witnessed in 2008 and 2020.

3 Literature Review

[Dick-Nielsen et al. \(2023\)](#) is the most important paper for this project, where the authors studied common factors that are used by academia to construct strategies on corporate bonds. They first outlined two types of datasets used by the industry:

1. Warga Dataset

This dataset runs from 1973-1998, and the Merrill Lynch/ICE data runs 1985-2021. However, this dataset only contains bid-ask quotes, and sometimes may not reflect prices which people are willing to trade on. Additionally, mere changes in the bid-ask quotes may reflect changes in bond returns without a change in real valuation of the bond.

2. TRACE Dataset

Instead of using quotes, this dataset relies on bond transaction data to keep a track of prices, and solves a lot of challenges faced by the Warga dataset. However, bond prices are not recorded when no transaction takes place, creating problems of its own.

Next, the authors found that most academic papers in this field dealt with dataset issues inconsistently, so the authors handled these inconsistencies and ran the models tested by these papers again. On a level playing ground, most factors lost their significance, leading to a replication error of all these papers.

Due to this, [Dick-Nielsen et al. \(2023\)](#) decided manually clean the TRACE dataset, following a robust approach to deal with outlier values, and calculate excess returns for all the corporate bonds. This cleaned dataset was uploaded by the authors on Christian Stolborg's personal website [Stolborg \(2023\)](#), and has been used for this project as well.

[Remillard, Nasri, and Ben-abdellatif \(2017\)](#) covers replication methods for financial indices, another relevant piece of literature for this project. The authors talk about two replication approaches:

1. Strong Replication

A method which involves constructing a portfolio of liquid assets, with the goal being tracking an actual index. Strong replication is further divided into two sub-categories:

- *Imitative/Naive Method*

The objective is to replicate the composition of the index. This method only works when the composition of the index we wish to replicate is known.

- *Factorial Method*

The objective here is to replicate the month-end returns of the index we wish to track, by investing in a portfolio of liquid assets.

2. Weak Replication

This method also creates a portfolio of liquid assets, but the objective here is to replicate the statistical properties of an index, or any other synthetic asset.

Then, the authors run a simple factorial-based strong replication exercise, where they construct the following linear problem:

$$R_t^* = \beta_t^\top \mathbf{R}_t + \varepsilon_t$$

where:

$$\begin{aligned} R_t^* &= \text{Returns of the target fund,} \\ \mathbf{R}_t &= \text{Returns of the factors (liquid assets),} \\ \beta_t &= \text{Weights associated with the factors.} \end{aligned}$$

and used linear regression and Kalman filters to solve it. Kalman filters performed better than the regression, and the authors mentioned the use of machine learning techniques to solve this problem better. This exact methodology is followed by this project as well.

4 Quantitative Methodology

As explained in the previous sections, the problem at hand is quite simple: there is a time-series of index returns, which the project intends to replicate (*index returns: target variable*). This is done through strong replication, which considers the individual corporate bond returns as features, and are used to predict index returns (*bond returns: features*). Using feature importance analysis, we can create portfolios of bonds which contribute highly towards the index returns, and would track the index. [Figure 3](#) below explains the process diagrammatically:

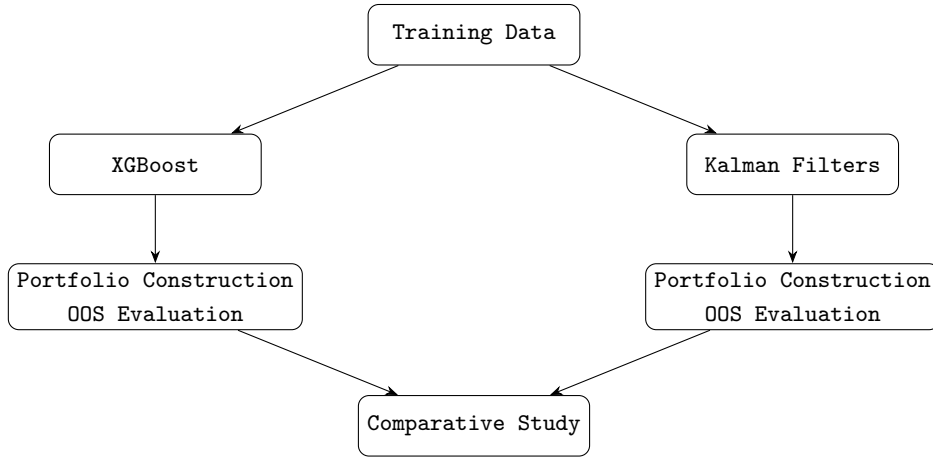


Figure 3: Functional Flow

[Formulas 1 and 2](#) display the problem mathematically:

$$\mathbf{R}_{t-24:t-1}^{\text{Index}} = \mathbf{X}_{t-24:t-1} \beta_{t-1} + \varepsilon_{t-1} \quad (1)$$

$$\hat{R}_t^{\text{Index}} = \mathbf{X}_t \beta_{t-1} \quad (2)$$

- $\mathbf{R}_{t-24:t-1}^{\text{Index}}$: 24×1 vector of index returns (train)

- $\mathbf{X}_{t-24:t-1}$: $24 \times N$ matrix of bond returns (train)
- β_{t-1} : $N \times 1$ vector of bond weights
- ε_{t-1} : 24×1 vector of residual errors
- $\mathbf{X}_t$: $1 \times N$ vector of bond returns (test)
- $\hat{R}_t^{\text{Index}}$: Predicted index return at time t

Python was used for implementation, the code for which can be found in [Appendix A2: Code](#).

4.1 Rolling Window Evaluation

The entire dataset contains 232 rows (months) of data, and the train-test splits have been done manually, with 24 rows (months) being the rolling window. The Out-of-sample (OOS) evaluation consists of using the portfolio weights constructed in the training window, and applying it to the bond returns of next month, imitating real-life decision making.

In order to make sure there is no look-ahead bias, the signals (weights) that get generated at each rolling window iteration, are lagged by one more month by default, before being acted upon.

- In-sample window size: 24 months
- Out-of-sample window size: 1 month
- Rolling step: 1 month

$$\begin{aligned} \text{In-sample window: } & [k : k + 24) \\ \text{Out-of-sample window: } & [k + 24 + 1] \\ & k = 0, 1, \dots, 207 \end{aligned}$$

The +1 in the OOS window signifies the additional time lag which has been introduced to combat look-ahead bias. [Table 1](#) below showcases the rolling window process:

Table 1: Rolling Window Logic

Loop	In-sample Window	Out-of-sample Window
1	[0 : 24)	[25]
2	[1 : 25)	[26]
3	[2 : 26)	[27]
$\vdots$	$\vdots$	$\vdots$
208	[207 : 231)	[232]

Do note that, the brackets [and (signify inclusive and exclusive indices respectively, consistent with Python syntax. Tracking error is used to measure how efficiently the portfolios are replicating the index. As noted by [Remillard, Nasri, and Ben-abdellatif \(2017\)](#), tracking error is defined in [Formula 3](#):

$$\text{Tracking Error} = \sqrt{\frac{1}{T} \sum_{t=1}^T (R_t^{\text{Portfolio}} - R_t^{\text{Index}})^2} \quad (3)$$

The models described below aim to reduce this tracking error at each iteration.

Since most bonds did not have maturities spanning the entire dataset, NaN values were recorded for bonds that expired / did not exist for specific dates. This created challenges for training the models, as NaNs are not handled well by most techniques. One of the reasons why XGBoost was chosen for this project was due to its innate ability to handle NaNs well.

Additionally, an NaN in the TRACE dataset might also signify no transaction activity on the bond, hence artificially imputing prices of bonds would have led to inaccurate estimates.

To combat this problem, a special function was created, to manually slice train-test splits, and get a subset of bonds which had a coverage of 80% in the training period, and existed during the test period. This ensured that a bond that was used in training the models, was available to be used in portfolio construction while testing and evaluating the models.

4.2 XGBoost

XGBoost (*eXtreme Gradient Boosting*) is an ensemble machine learning model that combines concepts of ensemble learning and gradient descent optimization. An ensemble model usually consists of combining multiple weak learners like shallow decision trees, and aggregating them to achieve stronger predictions. Boosting does this process sequentially, where new models try to reduce the errors committed by previous models.

In the case of XGBoost, the algorithm starts by initializing model M_0 with a constant prediction, like the mean of the target variable $\bar{y}$:

$$\hat{y}^{(0)} = \bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$$

Next, residuals of the predictions are calculated using a loss function. As this project is solving a regression problem, the loss function chosen is squared error loss. The residuals at iteration t are:

$$r_i^{(t)} = -\frac{\partial \mathcal{L}(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i^{(t-1)}} = y_i - \hat{y}_i^{(t-1)}$$

Then, a regression tree H_t is fit to these residuals, with an aim to improve previous' models prediction. The updated prediction is:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta \cdot H_t(x_i)$$

where η is the learning rate that scales the contribution of each new tree.

The process iterates this way, fitting new trees to sequential residuals, and accordingly updating the predictions:

$$\hat{y}_i^{(t+1)} = \hat{y}_i^{(t)} + \eta \cdot H_{t+1}(x_i)$$

The algorithm continues until a stopping criterion is met, like number of trees, depth of trees, etc. As these are hyperparameters which govern the effectiveness of the model, the value for each hyperparameter was chosen using Cross-Validation (CV) and GridSearch, ensuring robust results.

Overall, XGBoost is a very robust algorithm that captures non-linear dynamics, has built-in regularization measures that combat overfitting, and can be run on multiple cores at once, helping reduce computation times.

4.3 Kalman Filter

Kalman Filters come from Bayesian Econometrics, which says that getting the conditional probability of the parameters given the data is the best way of summarizing what has been learnt. In this context, according to the Bayes Theorem;

$$p(\theta_i | y_i) = \frac{p(y_i | \theta_i) \cdot p(\theta_i)}{p(y_i)}$$

posterior	$p(\theta_i y_i)$
marginal likelihood	$p(y_i \theta_i)$
prior	$p(\theta_i)$
marginal density of y_i	$p(y_i)$

Ignoring the marginal density of y_i implies:

$$p(\theta_i | y_i) \propto p(y_i | \theta_i) \cdot p(\theta_i)$$

Using this fundamental result, the parameters are assumed to be unobservable, and their dynamics are governed by the *Space / Transition Equation*. These unobservable entities are linked to the observable variables (returns) by the *Measurement Equation*. This system of Space and Measurement equations is called a *State-Space Model*, and Kalman Filter is a particular algorithm which solves this model in the linear and Gaussian case.

Formulas 4 and 5 are the State and Measurement equations respectively.

$$\beta_t = \beta_{t-1} + \eta_t, \quad \eta_t \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}) \tag{4}$$

$$y_t = \mathbf{x}_t^\top \beta_t + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, R) \tag{5}$$

where

$$\begin{aligned} \beta_t &\in \mathbb{R}^N : \text{latent portfolio weights at time } t \\ \mathbf{x}_t &\in \mathbb{R}^N : \text{vector of bond returns at time } t \\ y_t &\in \mathbb{R} : \text{observed index return at time } t \\ \eta_t &\sim \mathcal{N}(\mathbf{0}, \mathbf{Q}) : \text{state noise with covariance } \mathbf{Q} \\ \varepsilon_t &\sim \mathcal{N}(0, R) : \text{observation noise with variance } R \end{aligned}$$

Additionally,

$$\beta_1 \sim \mathcal{N}(\mathbf{0}, \mathbf{P}_0)$$

is the initial state prior where portfolio weights start off as 0, and $\mathbf{P}_0$ is the initial state covariance matrix. The weights follow a simple Random Walk (RW), as it has desirable properties like no mean-reversion, and previous weights are the best predictor of current weights. These properties integrate well with the project's environment, as a lot of bonds will mature overtime (no need for mean-reversion) and a lot of bonds will be bought/sold throughout (recent weights hold the most value).

5 Model Evaluation

First the model results were used to understand how many bonds were the portfolios holding overtime, and how did it compare with the bonds present in the index.

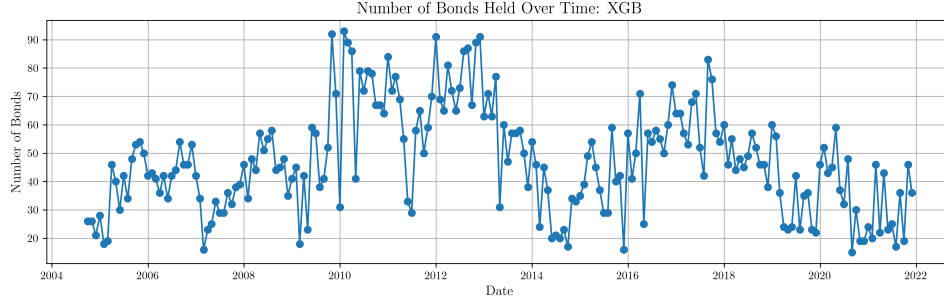


Figure 4: Bonds Held Overtime: XGB

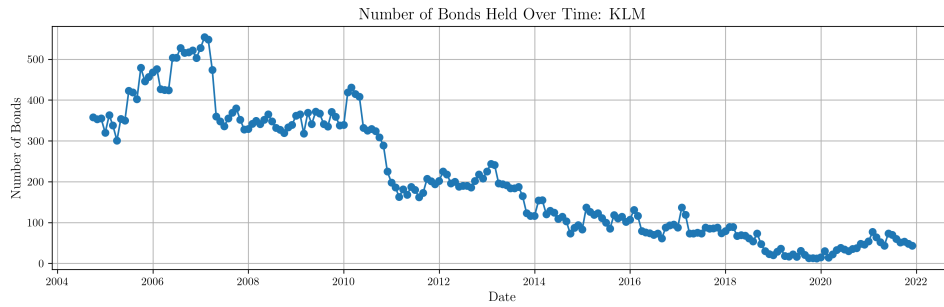


Figure 5: Bonds Held Overtime: Kalman

Figures 4 and 5 plot the bonds held overtime by both the models. XGB performed really well in this regard, as it held an average of 50 bonds overall, which is very impressive considering one training split had an average of 6000 investable bonds. The Kalman Filter held more bonds than XGB, but still impressively less than the number of bonds making up the index. However, the downward trend observed in Kalman bonds needs to be studied further.

The most optimal way of judging regression problems is by computing squared errors. This synergizes well with the project, as a famous regression error metric *Root Mean Square Error*, or *RMSE* is nothing but the tracking error that was mentioned before.

It is important to note that comparing the absolute in-sample and OOS values with each other is statistically dubious, as 24 values were used to calculate the in-sample tracking error, whereas only 1 value was used to calculate the OOS tracking error.

Table 2 summarize the error statistics of the weights generated by both the models. As observable, XGB had a better performance on average, as it recorded a monthly OOS tracking error of 0.58%, compared to the 0.75% of Kalman. Annually, these translate to 2.01% and 2.60% respectively.

The statistics in Table 3 paint the following picture: the errors from XGB are positively skewed, and have longer right-tails. XGB recorded more errors on the lower end, but has higher errors too, making it more volatile than Kalman weights. In general, XGB was more accurate at the cost of being volatile, and Kalman was smooth at the cost of being inaccurate.

Table 2: Tracking Error Summary Statistics (%)

	count	mean	std	min	25%	50%	75%	max
INS_TE_XGB	208	0.6858	0.5328	0.1934	0.3804	0.5245	0.7159	3.3722
OOS_TE_XGB	207	0.5846	1.1439	0.0138	0.1524	0.3203	0.6115	11.6097
INS_TE_KLM	208	1.1154	0.5059	0.3081	0.8138	1.0648	1.3022	2.6041
OOS_TE_KLM	207	0.7494	0.8808	0.0053	0.2119	0.4802	0.8721	6.8661

Table 3: Portfolio Returns Summary (% , annualized)

	Mean	Std_Dev	Skewness	Kurtosis	Correlation	Sharpe
Idx_Exc	4.0491	5.7406	-0.7661	8.9044	1.0000	0.2036
XGB_OOS_RET	4.8024	7.4929	0.5556	14.7901	0.8082	0.1850
KLM_OOS_RET	6.6699	8.6941	0.2476	8.4323	0.9354	0.2215

[Figure 13](#) plots the unconditional distribution of the tracking error for both models, which showcases this analysis. Surprisingly, XGB recorded the maximum tracking error as well, a difference that is studied in the following section.

5.1 Tracking Errors

As explained before, comparing the in-sample and OOS errors between the models makes more sense statistically.

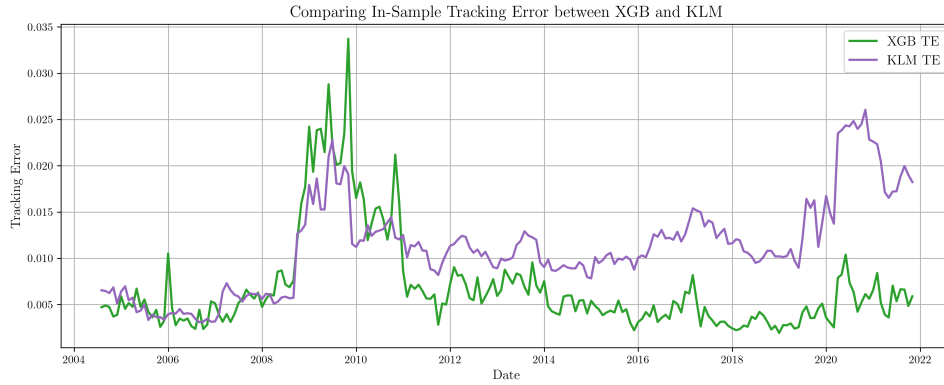


Figure 6: In-Sample Tracking Errors: XGB vs KLM

[Figure 6](#) plots the in-sample tracking errors of both models. Before 2008, the errors from Kalman were more stable, and also recorded lower peaks during the financial crisis. However, post-2008, Kalman consistently recorded higher errors on average, which coincided with Kalman buying lesser bonds overtime. The combination of lower bonds COVID meant Kalman errors jumped higher in 2020.

[Figure 12](#) also plots the in-sample and OOS errors for the models separately, which can be accessed in [Appendix A1: Plots](#).

[Figure 7](#) plots the OOS tracking errors of both models. A similar trend is observed, but Kalman's performance only start deteriorating from 2016, which strongly coincides with the model buying fewer bonds. This implies that Kalman's stability can be linked to using more bonds for

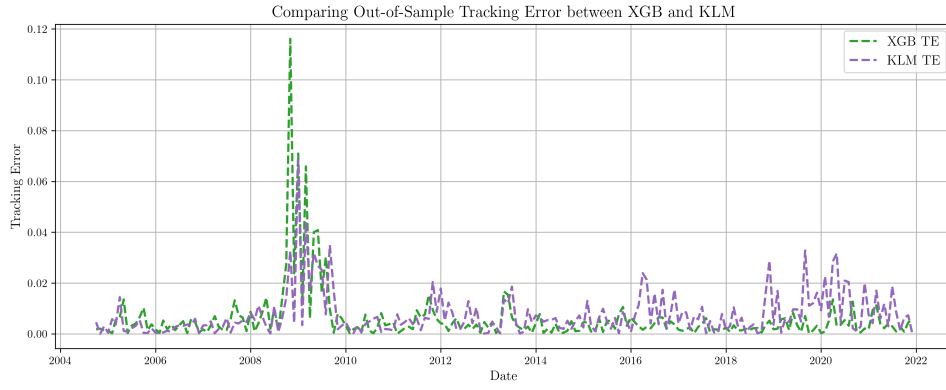


Figure 7: OOS Tracking Errors: XGB vs KLM

creating the replicating portfolio.

However, Kalman's performance is still impressive, given it only captured the linear dynamics. Overall, XGB's results prove more robust, as the non-linear dynamics were able to reduce the overall tracking error.

5.2 Correlation Analysis

Another important evaluation metric would be the correlation between the portfolios and the index.

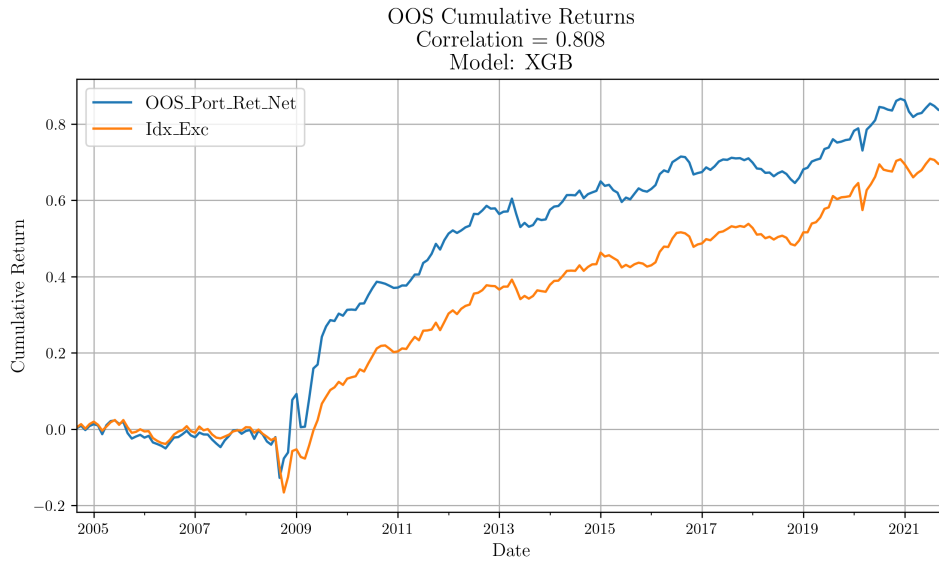


Figure 8: Correlation Analysis: XGB

Figure 8 plots the correlation between the index and XGB's portfolio. These returns are calculated net of transaction costs. As bonds are illiquid in nature, transaction costs of **10bps** are levied everytime the portfolio rebalances weights. XGB's portfolio tracked the index perfectly up until the crisis, and was shocked upwards and has stayed there since then.

Figure 9 plots the correlation between the Kalman portfolio and the index. The Kalman model has more correlation with the index, gave higher cumulative returns than the XGB model, and was also able to perfectly track the index up until the financial crisis hit.

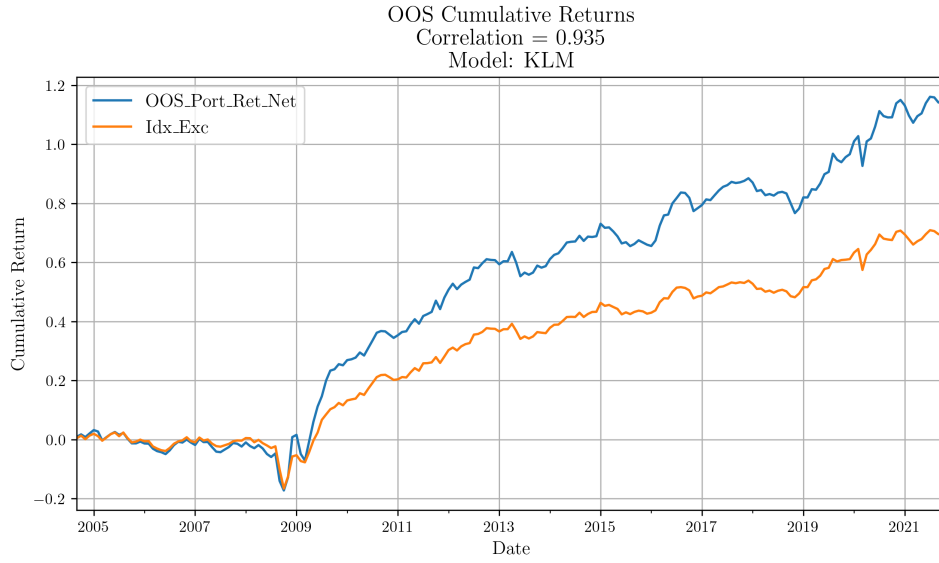


Figure 9: Correlation Analysis: Kalman

These results show the efficacy of the models, and the higher returns earned can be used to better market the ETF. Combined with the fact that both models were able to replicate the index so well while only investing in a fraction of the bonds available, displays the practical advantages of these tools, and can be seriously considered as ways to improve current offerings.

6 Conclusions

The main conclusions that can be derived from this project are:

1. Crisis Handling

Both models performed perfectly till 2008, but observed massive jumps in tracking error during the financial crisis. As these outlier values affected the training samples, the errors for next 2 years remained relatively high. Processing these outliers may yield stronger results.

2. XGB Performance

XGB performed better in terms of average OOS errors, and also bought fewer bonds thanks to strong regularization structures present within the model. However, XGB errors were more volatile and recorded higher peaks than Kalman errors.

3. Kalman Performance

Kalman cumulative returns deviated more from the index, but also presented higher correlation. This suggests that the Kalman portfolio can be combined with another instrument (eg: risk-free) to better match the cumulative returns of the index.

4. ETF Construction

As both models had high correlations with the index, higher cumulative returns, and bought fewer bonds, ETF managers could use these results to reduce transaction costs while improving ETF performance, making their products more attractive to investors.

5. Alternative Beta

The results from this project can also be used to create *Alternative Beta* strategies, where

passive products can be created to broadly track an index, and also gain exposure to specific market factors, earning higher returns than the index.

6.1 Challenges

The main challenges for this applied project were:

1. Handling NaNs

The biggest challenge was handling the NaN values in the TRACE dataset. It was difficult to ascertain whether NaN values were attributable to maturity or no transactions, yielding price imputing measures irrelevant. Dropping bonds with NaN values would only result in very highly-liquid bonds being available, defeating the point of this exercise. A function was developed to solve this, but it still remained a critical issue.

2. Monthly Frequency

It is very difficult for a model to learn about the bond's relationship with the index using only 24 past values. Daily granularity would have allowed the models to learn more complex relationships under the same training window, but would have come with computational costs.

3. Computational Capacity

The immense number of bonds made the exercise computationally challenging. Even though each bond only had 24 observations, at any given iteration, there were 3500–6000 investable bonds, translating to a high number of features, and robustly training the models consumed a lot of time.

References

- Bloomberg L.P. (n.d.). *Bloomberg Terminal*. <https://www.bloomberg.com/professional/solution/bloomberg-terminal/>. Accessed via Bloomberg Terminal, [June 2025].
- Dick-Nielsen, Jens et al. (Sept. 28, 2023). *Corporate Bond Factors: Replication Failures and a New Framework*. SSRN Electronic Journal. DOI: [10.2139/ssrn.4586652](https://ssrn.com/abstract=4586652). URL: <https://ssrn.com/abstract=4586652>.
- Remillard, Bruno, Bouchra R. Nasri, and Malek Ben-abdellatif (July 11, 2017). *Replication Methods for Financial Indexes*. DOI: [10.2139/ssrn.3000340](https://ssrn.com/abstract=3000340). URL: <https://ssrn.com/abstract=3000340>.
- Stolborg, Christian (2023). *Corporate Bond Factors Data Repository: Monthly Factor Excess Returns (1985–2021)*. Accessed via Christian Stolborg's personal site, data linked to "Corporate Bond Factors: Replication Failures and a New Framework". URL: <https://www.stolborg.com/data>.

Appendix A1: Plots

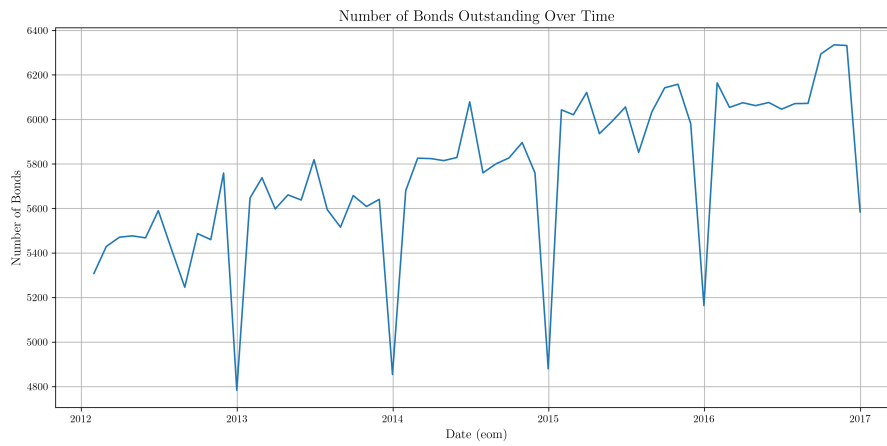


Figure 10: Corporate Bonds Outstanding (Sliced)

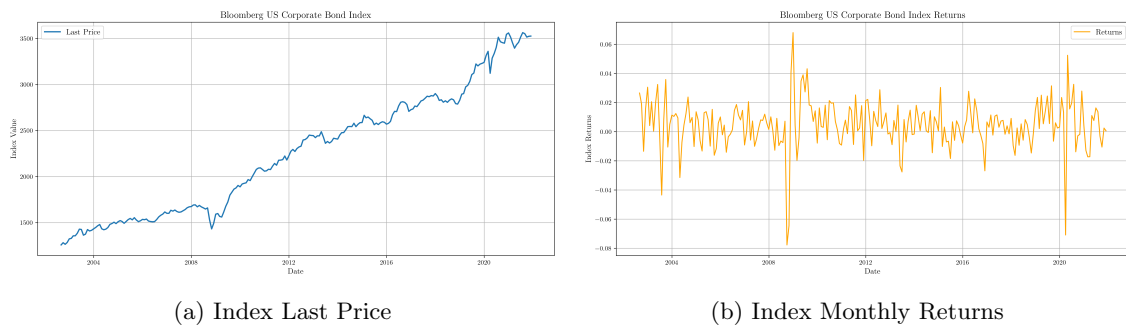


Figure 11: Bloomberg Index Plots

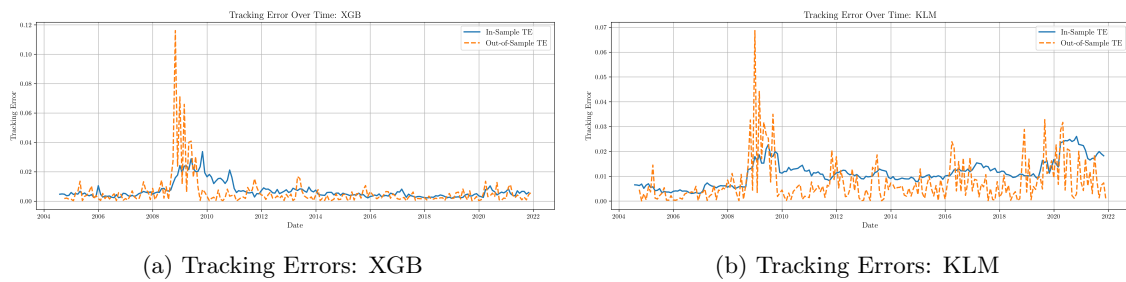


Figure 12: Tracking Errors Overtime

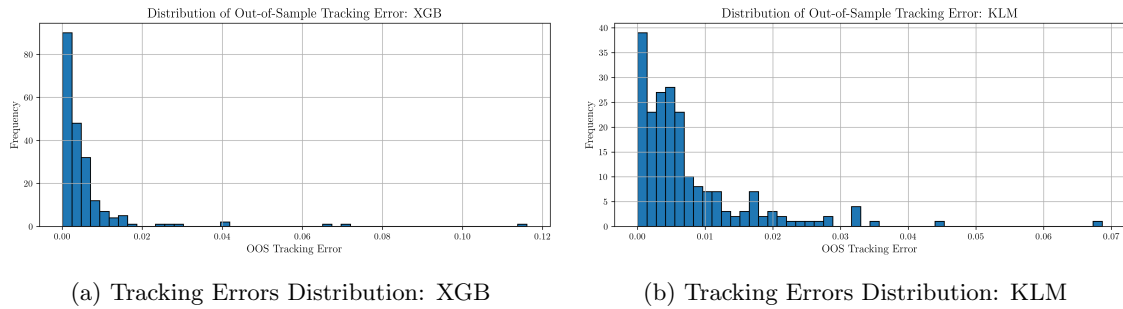


Figure 13: Tracking Errors Distribution

Appendix A2: Code

Listing 1: Cleaning Bonds Data

```
def cleaning_bonds_data (bonds_raw_data : pd.DataFrame) -> pd.DataFrame:
    """
    A function which performs all the data cleaning exercises for the
    corporate bonds
    dataset (TRACE) and outputs the processed dataframe for data
    exploration.
    """
    # 1) Filtering out High-Yield bonds
    output_df = bonds_raw_data[bonds_raw_data['rating_group'] != 'SG'].
        copy()

    # 2) Converting the EDM column to 'date' and into pandas datetime
        format
    output_df['eom'] = pd.to_datetime(output_df['eom'], format='%Y-%m-%d',
        , errors='raise')

    # 3) Sorting the resulting dataframe in a chronological order
    output_df.sort_values(by='eom', inplace=True)

    # 4) Dropping unnecessary columns
    output_df.drop(['permno', 'ret_text', 'yield', 'duration', '
        rating_group'],
        axis=1, inplace=True)
    output_df.rename(columns={'eom': 'Date'}, inplace=True)

    # 5) Resetting the index
    output_df.set_index('Date', inplace=True)

    # Returning the resulting dataframe
    return output_df
```

Listing 2: Cleaning Index Data

```
def cleaning_index_data (index_raw_data : pd.DataFrame,
    clean_tbill : pd.DataFrame,
    start_index : pd.Timestamp,
```

```

end_index : pd.Timestamp) -> pd.DataFrame:
"""
A function which performs all the data cleaning exercises for the
Bloomberg index data
(LUACTRUU) and outputs the processed dataframe for data exploration.
Function also calculates excess returns over T-Bill rate, and assumes
the the T-Bill
data which is being passed as input has been sliced with the same
dates as the bonds.
"""
# Creating output dataframe and neccessary variables
interim_df = index_raw_data.copy()
first_col_name = index_raw_data.columns[0]
index_value_col = index_raw_data.columns[3]

# 1) Renaming Date column and setting it as the index
interim_df.rename(columns={first_col_name:'Date',
                           index_value_col:first_col_name}, inplace=
                           True)
interim_df['Date'] = pd.to_datetime(interim_df['Date'], format="%d-%m
-%Y", errors='raise')
interim_df.set_index('Date', inplace=True)

# 2) Resampling the dataframe to match Monthly Frequency
interim_df = interim_df.resample("ME").last()

# 3) Calculating Returns
interim_df['Returns'] = (interim_df['PX_LAST'] / interim_df['PX_LAST'
].shift(1)) - 1

# 4) Slicing the dataframe to match the bonds dataset
interim_df = interim_df[interim_df.index >= start_index]
interim_df = interim_df[interim_df.index <= end_index]

# 4) Creating final dataframe with relevant columns
output_df = pd.DataFrame(index = interim_df.index)
output_df['Idx_Price'] = interim_df['PX_LAST']
output_df['Idx>Returns'] = interim_df['Returns']
output_df['Idx_Exc'] = interim_df['Returns'] - clean_tbill['DGS1M0']
output_df['Idx_MCap'] = interim_df['CUR_MKT_CAP']
output_df['Idx_Count'] = interim_df['COUNT_INDEX_MEMBERS']

# Returning the resulting dataframe
output_df.dropna(inplace=True)
return output_df

```

Listing 3: Cleaning T-Bill Data

```

def cleaning_tbill_data (tbill_raw_data : pd.DataFrame,
                        start_index: pd.Timestamp,
                        end_index : pd.Timestamp) -> pd.DataFrame :
"""
A function which performs all the data cleaning exercises for the raw

```

```

1-M T-Bill
rate (DGS1M0) which has been pulled from Fred.
"""
# 1) Resampling to get monthly frequency
output_df = tbill_raw_data.resample('ME').last()

# 2) Converting Annual Percent to Monthly Decimal
output_df['DGS1M0'] = output_df['DGS1M0'] / 1200

# 3) Slicing dataframe to match the bond dataset
output_df = output_df[output_df.index >= start_index]
output_df = output_df[output_df.index <= end_index]

# 4) Changing the name of the index
output_df.index.name = 'Date'

# Returning final dataframe
output_df.dropna(inplace=True)
return output_df

```

Listing 4: Cleaning Pipeline

```

def cleaning_pipeline (bonds_raw_data : pd. DataFrame,
                      index_raw_data : pd.DataFrame,
                      tbill_raw_data : pd.DataFrame):
    '''
    Main pipeline function which performs all the data cleaning exercises
    , and
    returns a dictionary containing all the relevant variables.
    '''
    # Performing data cleaning for Bonds dataset
    clean_bonds_data = cleaning_bonds_data (bonds_raw_data)
    pivot_data = pivot_bonds_data (clean_bonds_data)
    processed_bond_ret = pivot_data['bond_returns']
    processed_bond_mcap = pivot_data['bond_mcap']

    # Performing data cleaning for TBill dataset
    start_date = processed_bond_ret.index[0]
    end_date = processed_bond_ret.index[-1]
    clean_tbill = cleaning_tbill_data(tbill_raw_data, start_date,
                                     end_date)

    # Performing data cleaning for Bloomberg Index dataset
    processed_index = cleaning_index_data (index_raw_data, clean_tbill,
                                           start_date, end_date)

    # Returning all the variables
    return {
        'clean_bonds_long' : clean_bonds_data,
        '1m_tbill' : clean_tbill,
        'processed_bonds_ret' : processed_bond_ret,
        'processed_bonds_mcap' : processed_bond_mcap,
        'processed_index' : processed_index
    }

```

Listing 5: Converting Bonds Structure

```
def pivot_bonds_data (clean_bonds_data : pd.DataFrame) :
    """
    A function which takes as input the processed bonds dataset, and
    changes the structure of this
    dataframe such that the rows are just the dates, and each date has
    all available bonds as
    the columns (features). This structure will make it easier to feed
    this data into the models.
    Returns a dictionary of wide-format dataframes where each key
    corresponds to a bond-level feature.
    Column names will be the CUSIP of each bond.
    Also drop those bonds who have 1 or less non-null values
    """
    # Creating necessary variables
    interim_df = clean_bonds_data.reset_index()

    # Creating dataframe with bond returns
    pivot_bond_returns = interim_df.pivot(index='Date', columns='cusip',
        values='ret_exc')

    # Filtering out bonds with 1 or less valid observations
    drop_list = pivot_bond_returns.columns[pivot_bond_returns.count() <=
        1].tolist()

    # Dropping irrelevant rows and columns
    pivot_bond_returns.drop(columns=drop_list, inplace=True)
    pivot_bond_returns = pivot_bond_returns.iloc[1:]

    # Creating another dataframe with bond MCap
    pivot_bond_mcap = interim_df.pivot(index='Date', columns='cusip',
        values='market_value')
    pivot_bond_mcap.drop(columns=drop_list, inplace=True)

    return {
        'bond_returns' : pivot_bond_returns,
        'bond_mcap' : pivot_bond_mcap
    }
```

Listing 6: Bond Universe Selection

```
def select_bond_universe (train_features: pd.DataFrame,
    test_features: pd.DataFrame,
    coverage_percent: float=0.8):
    """
    A function that returns a DataFrame including bonds that are
    investable based on availability.
    Availability, in this context, means that a bond must have return
    values at least once in the
    test set, and should atleast have coverage_percent returns in
    training set.
    """
```

```

# Creating copies
train_df = train_features.copy()
test_df = test_features.copy()

# Creating necessary variables
n_rows = train_df.shape[0]

# Dropping bonds which do not have any returns in the dataframes
train_drop_list = train_df.columns[train_df.count() == 0].to_list()
test_drop_list = test_df.columns[test_df.count() == 0].to_list()
train_df = train_df.drop(columns=train_drop_list)
test_df = test_df.drop(columns=test_drop_list)

# Getting list of train bonds which have minimum coverage
coverage_ratio = train_features.count() / n_rows
train_cols = coverage_ratio[coverage_ratio >= coverage_percent].index

# Using sets to get bonds common to both train and test datasets
train_cols = set(train_cols)
test_cols = set(test_df.columns)

# Creating investable list using set intersection
investable_list = list(train_cols & test_cols)

# Getting the output dataframes with the selected bonds
# Filling nan values in the train dataset with 0.0
output_train = train_df[investable_list].copy()
output_train.fillna(0.0, inplace=True)
output_test = test_df[investable_list].copy()

# Returning the outputs
return {
    "investable_train" : output_train,
    'investable_test' : output_test
}

```

Listing 7: XGBoost Model

```

def xgboost_model (bond_returns: pd.DataFrame,
                   index_returns: pd.DataFrame,
                   param_grid: dict = None,
                   cv: int = 5,
                   random_state: int = 42,
                   shap_threshold_quantile: float = 0.0):
    """
    A function that runs an XGBoost model for a given slice of training
    data, tunes hyperparameters
    using cross-validation, and uses SHAP (SHapley Additive exPlanations)
    for feature importance.
    Portfolio weights are computed using the SHAP values.
    """
    # Creating necessary variables
    features_df = bond_returns.reindex(index_returns.index)

```

```

target_df = index_returns.squeeze()

# Setting the parameter grid
if param_grid is None:
    param_grid = {
        'n_estimators' : [50, 100],
        'max_depth' : [3, 4],
        'learning_rate' : [0.05, 0.1],
        'subsample' : [0.8, 1.0]
    }

# Fitting XGBoost with CV
base_xgb = XGBRegressor(objective = 'reg:squarederror',
                        random_state = random_state)

# Conducting grid search
grid_search = GridSearchCV(estimator = base_xgb,
                           param_grid = param_grid,
                           scoring = 'neg_mean_squared_error',
                           cv=cv,
                           n_jobs=-1)

# Fitting the model with best hyperparams
grid_search.fit(features_df, target_df)
tuned_xgb = grid_search.best_estimator_

# SHAP Analysis
explainer = shap.TreeExplainer(tuned_xgb, features_df)
shap_values = explainer(features_df)
shap_importance = np.abs(shap_values.values).mean(axis=0)
shap_series = pd.Series(shap_importance, index=features_df.columns)

# Select bonds above threshold
threshold = shap_series.quantile(shap_threshold_quantile)
selected_bonds = shap_series[shap_series > threshold].index.tolist()

# Normalize weights
weights = shap_series[selected_bonds].copy()
#weights = shap_series
weights /= weights.sum()

# Returning output variables
return {
    'xgb_model': tuned_xgb,
    'portfolio_weights': weights,
    'selected_bonds': selected_bonds,
    'shap_values': shap_series,
    'best_params': grid_search.best_params_
}

```

Listing 8: Kalman Model

```

def kalman_model(bond_returns: pd.DataFrame,
                 index_returns: pd.DataFrame,

```

```

        state_cov_scale: float = 1e-5,
        obs_cov_scale: float = 1e-3):
'''
Fits a time-varying linear model using the Kalman Filter.
This replicates index_returns using bond_returns with dynamically
estimated weights.

Inputs:
- bond_returns: DataFrame of shape (T, N) = returns of N bonds over T
  time steps.
- index_returns: Series of shape (T,) = target excess return.
- state_cov_scale: scaling parameter for the state covariance (
  controls weight flexibility).
- obs_cov_scale: scaling parameter for the observation noise (
  controls fit to target).
'''

# Align inputs
X = bond_returns.values # shape (T, N)
y = index_returns.values # shape (T,)

n_timesteps, n_bonds = X.shape

# --- Kalman Filter Setup ---

# Observation matrix Z_t = X_t
# At each time t, we observe: y_t = X_t @ beta_t + epsilon_t

# Design matrix: time-varying observation matrices
observation_matrices = X.reshape((n_timesteps, 1, n_bonds)) # shape
(T, 1, N)

# Transition matrix: beta_t = beta_{t-1} + eta_t, so transition
# matrix is identity
transition_matrices = np.eye(n_bonds)

# Observation covariance: scalar or 1x1 matrix (assumes homoscedastic
# error)
observation_covariance = obs_cov_scale * np.eye(1)

# State (weight) covariance: controls how much beta_t is allowed to
# vary
transition_covariance = state_cov_scale * np.eye(n_bonds)

# Initial state estimate (weights)
initial_state_mean = np.zeros(n_bonds)

# Initial state covariance
initial_state_covariance = np.eye(n_bonds)

# --- Fit Kalman Filter ---
kf = KalmanFilter(
    transition_matrices=transition_matrices,

```

```

        observation_matrices=observation_matrices,
        transition_covariance=transition_covariance,
        observation_covariance=observation_covariance,
        initial_state_mean=initial_state_mean,
        initial_state_covariance=initial_state_covariance
    )

    # Estimate weights (beta_t) over time using EM algorithm and Kalman smoothing
    state_means, state_covs = kf.smooth(y) # state_means shape: (T, N)

    # Final estimated weights (most recent step)
    final_weights = pd.Series(state_means[-1], index=bond_returns.columns)
    final_weights = final_weights[final_weights.abs() > 0.001] # remove negligible weights
    final_weights /= final_weights.sum() # normalize

    # Return results
    return {
        'portfolio_weights': final_weights,
        'weight_path': pd.DataFrame(state_means, index=bond_returns.index,
                                     columns=bond_returns.columns)
    }

```

Listing 9: Tracking Error Calculation

```

def te_eval (train_features: pd.DataFrame,
             test_features: pd.DataFrame,
             train_target: pd.DataFrame,
             test_target: pd.DataFrame,
             model_dict):
    """
    A function that calculates the in-sample and out-of-sample tracking
    error of the weights
    computed by the respective model.
    All the inputs (except model_dict) are in DataFrame format, with
    dates as rows and bond cusip
    as columns respectively.
    """
    # Creating necessary variables
    weights = model_dict['portfolio_weights'].to_frame().T
    # weights now has structure of columns being cusip, and the values being weights
    # which has been assigned to each bond (cusip) which is used to create portfolio
    bonds_list = weights.columns.to_list()
    sliced_train_features = train_features[bonds_list].copy()
    sliced_test_features = test_features[bonds_list].copy()

    # Calculating OOS TE for the portfolio
    oos_weighted_returns = (sliced_test_features.values @ weights.values.T)[0][0] # Using matrix multiplication

```

```

# As we only have one value in OOS (1 month), the average and sqrt
are not required
oos_te = np.sqrt(((test_target.values[0][0] - oos_weighted_returns)
    **2).mean()) # Both are now in np.array format

# Calculating in-sample TE for the portfolio
# Creating a new dataframe that consists of portfolio weights for
each day
weights_df = pd.DataFrame(
    np.tile(weights.values, (len(sliced_train_features), 1)),
    index = sliced_train_features.index,
    columns = weights.columns
)

# Calculating weighted returns
ins_weighted_returns = (weights_df * sliced_train_features).sum(axis
    =1).to_frame()
ins_weighted_returns.columns = train_target.columns.to_list()

# Calculating tracking error
ins_te = np.sqrt(((train_target - ins_weighted_returns)**2).mean().
    values[0])

# Returning calculated TE values
return ins_te, oos_te

```

Listing 10: Rolling Window Framework

```

def rolling_framework (bond_returns : pd.DataFrame,
    index_returns : pd.DataFrame,
    min_cover: float=0.8,
    rolling_window : float = 24):

    '''
    The main rolling window function, which performs rolling-window
    evaluation for all the
    models which are passed as a list to the function. Also constructs
    the portfolios at
    each step, and computes OOS tracking-error estimates.
    Inputs:
    bond_returns: DataFrame having rows as dates, and columns as bond
    returns.
    index_returns: DataFrame having rows as dates, Index values as
    columns.
    '''

    # Making sure bond and index rows match
    target_df = index_returns['Idx_Exc'].to_frame()
    features_df = bond_returns.reindex(target_df.index)

    # Creating necessary variables
    # XGB Output variables
    output_weights_xgb = pd.DataFrame(0.0, index=target_df.index, columns
        =features_df.columns)
    output_te_xgb = pd.DataFrame(index=target_df.index, columns=['

```

```

    INS_TE_XGB', 'OOS_TE_XGB'])

# Kalman Output variables
output_weights_kalman = pd.DataFrame(0.0, index=target_df.index,
    columns=features_df.columns)
output_te_kalman = pd.DataFrame(index=target_df.index, columns=['
    INS_TE_KLM', 'OOS_TE_KLM'])

# Running the main rolling loop, and using models to construct
# portfolios
for i in range(len(bond_returns) - rolling_window):
    # Getting train and test splits for bond returns
    sliced_bonds_train = features_df.iloc[i : (i+rolling_window)]
    sliced_bonds_test = features_df.iloc[[i+rolling_window]]

    # Filtering bonds to get investable universe
    investable_dict = select_bond_universe(sliced_bonds_train,
                                           sliced_bonds_test,
                                           min_cover)
    train_returns = investable_dict['investable_train']
    test_returns = investable_dict['investable_test']

    # Getting train and test splits for index returns
    train_index = target_df.iloc[i : (i+rolling_window)]
    test_index = target_df.iloc[[i+rolling_window]]

    # Running XGBoost
    xgb_model = xgboost_model(bond_returns = train_returns,
                              index_returns = train_index)
    xgb_weights = xgb_model['portfolio_weights']
    xgb_bonds = xgb_model['selected_bonds']

    # Evaluating XGB Tracking Error
    ins_te_xgb, oos_te_xgb = te_eval(train_returns, test_returns,
                                      train_index, test_index,
                                      xgb_model)

    # Appending results to output DataFrames
    output_weights_xgb.loc[target_df.index[i + rolling_window],
        xgb_bonds] = xgb_weights
    output_te_xgb.loc[target_df.index[i + rolling_window-1], ['
        INS_TE_XGB']] = ins_te_xgb
    output_te_xgb.loc[target_df.index[i + rolling_window], ['
        OOS_TE_XGB']] = oos_te_xgb

    # Running Kalman Model
    kalman_dict = kalman_model(bond_returns = train_returns,
                              index_returns = train_index)
    kalman_weights = kalman_dict['portfolio_weights']
    kalman_bonds = kalman_weights.to_frame().T.columns.to_list()

    # Evaluating Kalman Tracking Error
    ins_te_kalman, oos_te_kalman = te_eval(train_returns,

```

```

        test_returns,

                                train_index, test_index,
                                kalman_dict)

    # Appending results to output DataFrames
    output_weights_kalman.loc[target_df.index[i + rolling_window],
        kalman_bonds] = kalman_weights
    output_te_kalman.loc[target_df.index[i + rolling_window-1], ['
        INS_TE_KLM']] = ins_te_kalman
    output_te_kalman.loc[target_df.index[i + rolling_window], ['
        OOS_TE_KLM']] = oos_te_kalman

    # Dropping initial training rows from output variables
    output_weights_xgb = output_weights_xgb.iloc[rolling_window:]
    output_te_xgb = output_te_xgb.iloc[rolling_window-1:]

    output_weights_kalman = output_weights_kalman.iloc[rolling_window:]
    output_te_kalman = output_te_kalman.iloc[rolling_window-1:]

    # Dropping those bonds which were not used in portfolio construction
    invested_mask_xgb = (output_weights_xgb != 0.0).any()
    output_weights_xgb = output_weights_xgb.loc[:, invested_mask_xgb]

    invested_mask_kalman = (output_weights_kalman != 0.0).any()
    output_weights_kalman = output_weights_kalman.loc[:,
        invested_mask_kalman]

    return {
        'xgb_weights': output_weights_xgb,
        'xgb_te': output_te_xgb,
        'kalman_weights': output_weights_kalman,
        'kalman_te': output_te_kalman
    }
}

```

Listing 11: Shifting OOS Weights

```

def update_oos (bond_returns: pd.DataFrame,
                index_returns: pd.DataFrame,
                model_weights: pd.DataFrame,
                model_te: pd.DataFrame,
                shift_value: int=1):
    """
    A function that recalculates the OOS Tracking-Error for a given model
    . Outputs a dataframe with
    the same in-sample tracking errors, but updated out-of-sample (OOS)
    tracking errors.
    """
    # Shifting the weights (signals) by the shift value
    weights_shifted = model_weights.shift(shift_value).dropna()

    # Updating Index returns to match shifted weights
    index_shifted = index_returns['Idx_Exc'].loc[weights_shifted.index].
        to_frame()

```

```
# Updating Bond returns to match the shifted weights
bonds_shifted = bond_returns.loc[weights_shifted.index,
                                  weights_shifted.columns]

# Calculating Updated portfolio returns
updated_port_ret = ((bonds_shifted * weights_shifted).sum(axis=1)).
    to_frame()
updated_port_ret.columns = index_shifted.columns.to_list()

# Calculating OOS Tracking Error
updated_oos = np.sqrt(((index_shifted - updated_port_ret)**2))

# Creating output variable
output_te = pd.DataFrame(index=model_te.index)
first_colname = model_te.columns[0]
output_te[first_colname] = model_te[first_colname]

# Appending the updated OOS to this output variable
output_te = output_te.join(updated_oos, how='left')
output_te.columns = model_te.columns.to_list()

# Returning the output variable
return {
    'weights_shifted': weights_shifted,
    'updated_te': output_te
}
```